Algorithms Portfolio

Curated algorithmic solutions with correctness and complexity analysis

---

Ryan McMillan

MCompSci (cand.) — University of New England

C++ primary • Python secondary

January 17, 2026

License

Contents

1 Introduction

This document is a curated portfolio of algorithmic solutions. Each selected problem includes:

- clear problem restatements,
- key invariants / observations,
- pseudocode,
- correctness sketches,
- complexity analysis,
- C++ (primary) and Python (secondary) implementations.

The focus is on clarity, correctness, and reasoning.

1.1 How to Use This Book

Problems include an optional **Author's Thinking** block, the authors plain language thinking and problem framing, which is excluded from exemplar builds.

Use this book to understand how to formally reason with and structure algorithm and data structures problems.

2 Arrays & Hashing

2.1 Demo 0000: Triangular Number (Pipeline Test)

Problem. Given an integer $n \geq 0$, compute

$$T(n) = \sum_{i=1}^{n} i.$$

Return $T(n)$.
Key idea. Use the closed-form identity

$$T(n) = \frac{n(n+1)}{2}.$$

This avoids iteration and directly demonstrates asymptotic reasoning.
Algorithm. Compute $\frac{n(n+1)}{2}$ using integer arithmetic.
Correctness sketch. We use the classical identity:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

Therefore the algorithm returns exactly $T(n)$ for all $n \geq 0$.
Complexity. Time: $O(1)$, Space: $O(1)$. For comparison, the iterative approach

$$\sum_{i=1}^{n} i$$

runs in $O(n)$ time and $O(1)$ space. As $n \to \infty$, the closed-form approach dominates asymptotically.

```cpp
#include <bits/stdc++.h>
using namespace std;

/*
  Demo problem for the portfolio PDF pipeline.

  Task: Given n >= 0, compute sum_{i=1..n} i = n(n+1)/2.

  Purpose: Exercise code inclusion and asymptotic discussion.
*/

long long triangular_number(long long n) {
    // O(1) time, O(1) space
    return n * (n + 1) / 2;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    long long n;
    if (!(cin >> n)) return 0;
    cout << triangular_number(n) << "\n";
    return 0;
}
```

Listing 1: Reference implementation (C++) for the triangular number problem.